



天津大学
Tianjin University



Accelerating LLM Inference Throughput via Asynchronous KV Cache Prefetching

Yanhao Dong¹, Yubo Miao¹, Weinan Li¹, Xiao Zheng¹, Chao Wang¹, Feng Lyu²

¹Alibaba Cloud

²School of Computer Science and Engineering, Central South University, Changsha, China

Background: vLLM's native XFormers backend

3 critical performance bottlenecks:

- **Suboptimal GPU Utilization**
- **Poor Cache Hit Rate**
- **Persistent Warp Stall**

Metric	Value
Compute Throughput	23.35%
Memory Throughput	47.10%
L1 Cache Hit Rate (Load)	0.75%
L2 Cache Hit Rate (Load)	0.06%
Cycles Per Instruction	27.68 cycles
Stall Long Scoreboard	21.34 cycles

Table 1: Hardware performance metrics of the attention kernel in vLLM's XFormers backend collected using a single NVIDIA H20 GPU running the Llama2-7B model with output tokens set to 4096 and batch size of 64.

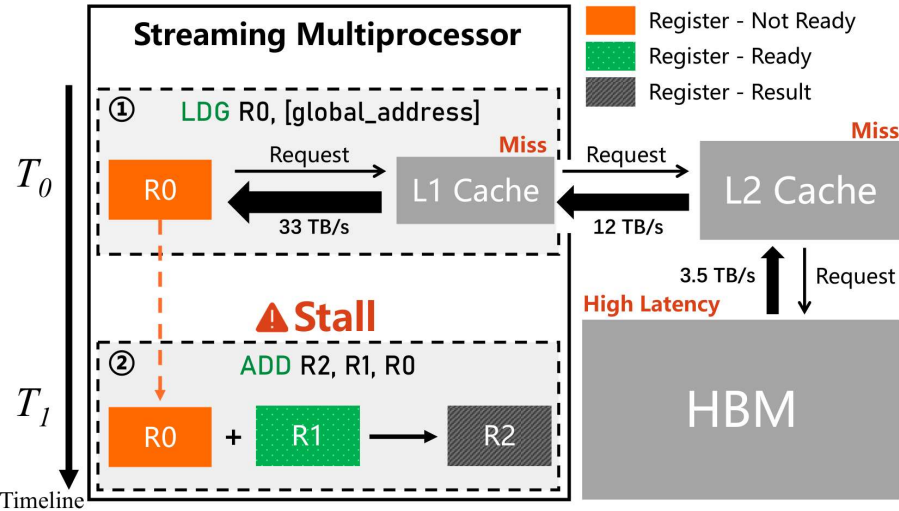


Figure 1: Schematic illustration of Stall Long Scoreboard event in NVIDIA GPU.

Asynchronous KV Cache prefetching

- Leverages idle memory bandwidth during compute cycles to prefetch KV Blocks into L2 cache, enabling direct cache hits and hiding HBM latency via compute-transmission overlap.

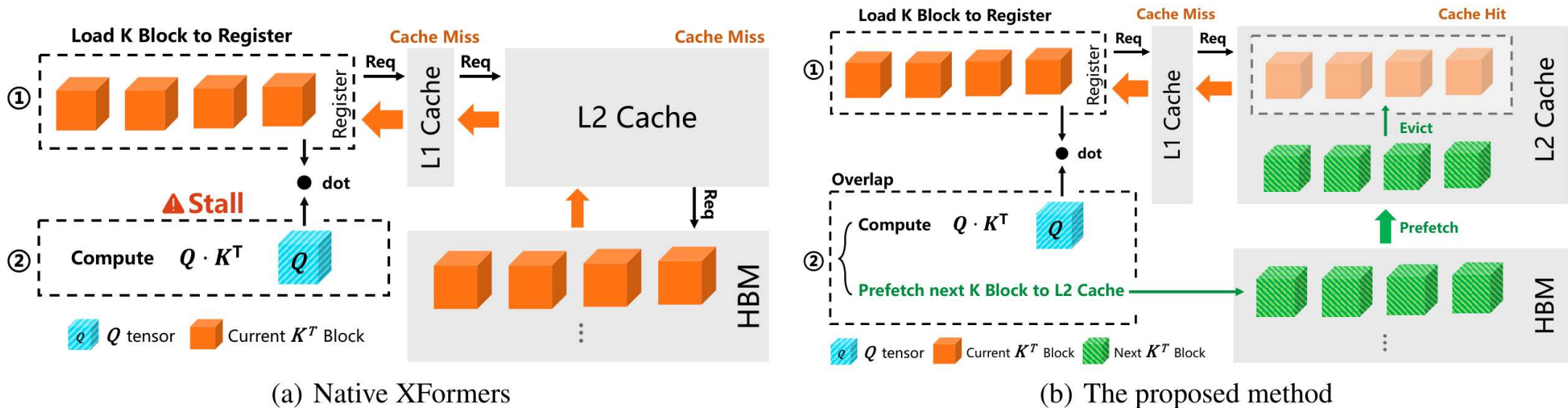


Figure 2: $Q \cdot K^T$ computation flow in a single iteration for native XFormers and the proposed method, illustrated with a thread block configuration containing 4 warps.

Asynchronous KV Cache prefetching

- leverages GPU's **asynchronous prefetch capability** to proactively load the next iteration's required K/V Blocks from high-latency HBM into L2 Cache.
- **the idle memory bandwidth's** capacity to efficiently accommodate additional data transfer demands
- concurrent execution of $Q \cdot K^T$ and K Block prefetching enables **computation-to-memory overlap**

Algorithm 1: Prefetching K Block to L2 Cache.

Input: Block table: $\mathbf{bt}$; Warps number: $\mathbf{w}$;
Block index range: $[s, e)$.

```
1: Let  $block\_idx = s$ ;  
2: while  $block\_idx < e$  do  
3:   Lookup  $\mathbf{bt}[block\_idx] \rightarrow k\_ptr$ ; // Current Block  
4:   Load K Block from  $k\_ptr$  to registers;  
5:   if  $block\_idx + \mathbf{w} < e$  then  
6:     Lookup  $\mathbf{bt}[block\_idx + \mathbf{w}] \rightarrow next\_k\_ptr$ ;  
       // Next Block  
7:     Perfetch K Block from  $next\_k\_ptr$  to L2 Cache;  
8:   end if  
9:   Compute  $QK^T$  for current K block;  
10:   $block\_idx \leftarrow block\_idx + \mathbf{w}$ ;  
11: end while
```

Asynchronous KV Cache prefetching

- GPU Prefetch Interface
 - NVIDIA CUDA introduces an L2 cache-oriented asynchronous prefetch mechanism through the PTX instruction `cp.async.bulk.prefetch.L2`
 - GPUs based on the **Hopper architecture**(e.g., H100/H20)
- **Quantitatively** analyzes the performance improvement limits
 - The memory footprint of a **single block** (b denotes the byte size per model parameter, d_h represents the dimension of a single attention head, and T_{block} indicates the number of tokens contained in each block).

$$M_{block} = b \cdot d_h \cdot T_{block}$$

- The **total memory** footprint (thread block dimension $[N_{thread}, 1, 1]$ and grid dimension $[H, B, 1]$)

$$M_{total} = M_{block} \cdot \frac{N_{thread}}{32} \cdot H \cdot B$$

Experimental Setup

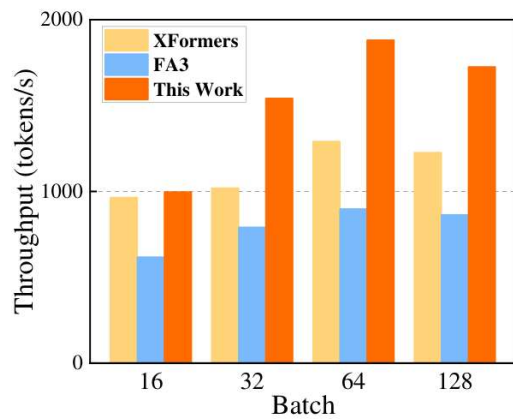
- **Hardware:**
 - NVIDIA H20 GPUs (60MB L2 cache, 4.0TB/s HBM).
- **Models:**
 - Llama2-7B (MHA), Llama3-8B, Qwen2.5-7B/14B (GQA), all in FP16.
- **Baselines:**
 - vLLM's XFormers and FlashAttention-3.

Results: Attention Kernel Performance

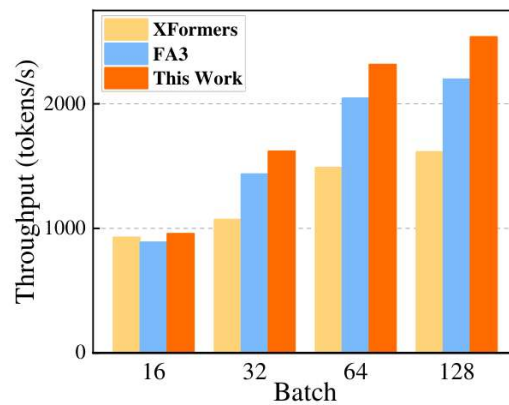
Metric	Llama2-7B		Llama3-8B		Qwen2.5-7B		Qwen2.5-14B	
	XFormers	This Work	XFormers	This Work	XFormers	This Work	XFormers	This Work
Duration (us)	293.47	159.14	226.53	119.90	232.03	107.71	272.19	143.94
Compute Throughput (%)	23.35	48.22	29.61	61.15	25.29	60.95	30.52	63.49
Memory Throughput (%)	47.10	86.88	23.44	66.96	20.05	63.50	24.16	68.44
L2 cache Hit Rate (%)	0.06	<u>43.70</u>	38.35	<u>73.01</u>	55.90	<u>82.66</u>	51.43	<u>77.11</u>
Cycles Per Instruction (cycle)	27.68	9.28	21.42	7.32	22.68	7.36	20.94	7.24
Stall Long Scoreboard (cycle)	21.34	<u>4.13</u>	16.66	<u>2.05</u>	16.38	<u>1.87</u>	16.17	<u>1.96</u>
Speedup	1.84×		1.89×		2.15×		1.89×	

Table 3: Attention kernel performance comparison across models using native XFormers backend vs. proposed method. Batch size and output tokens are taken as 64 and 4K, respectively. Models are run on an NVIDIA H20 GPU.

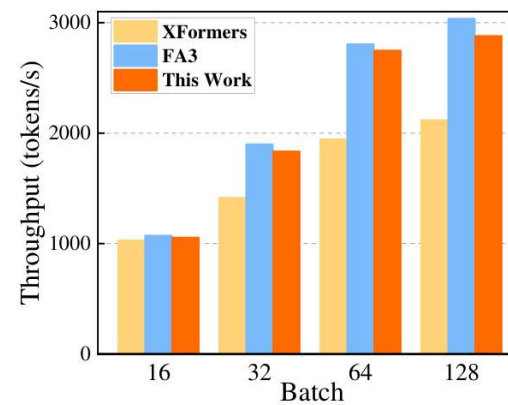
Results: Single-GPU end-to-end inference throughput



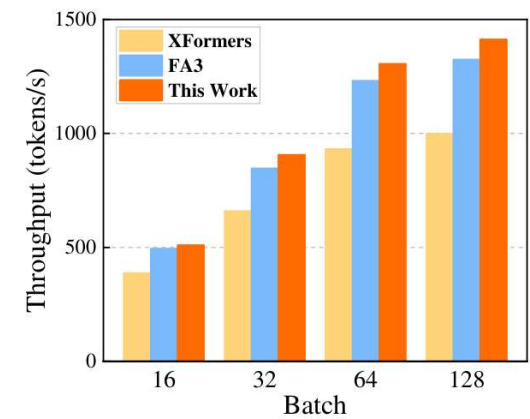
(a) Llama2-7B (32Q:32KV)



(b) Llama3-8B (32Q:8KV)



(c) Qwen2.5-7B (28Q:4KV)



(d) Qwen2.5-14B (40Q:8KV)

Figure 3: Single-GPU end-to-end inference throughput comparison across backends with fixed 2048 output tokens on H20.

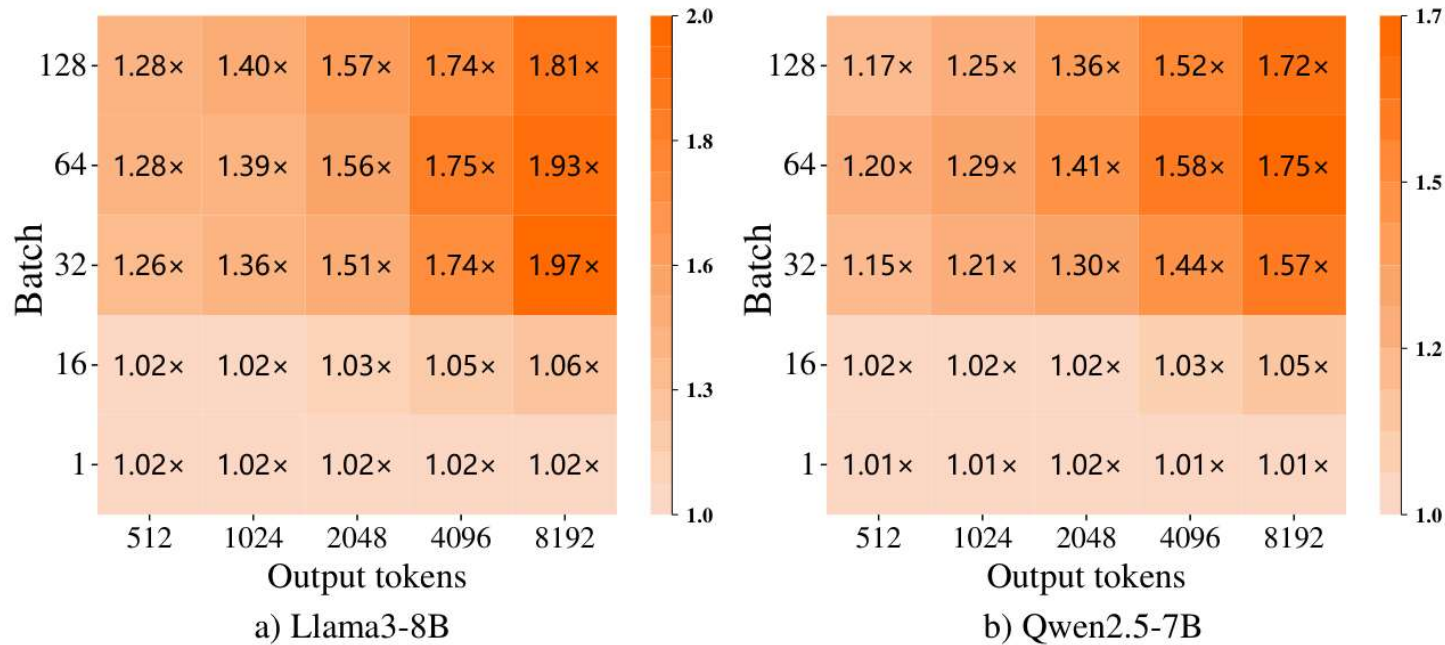


Figure 4: Speedups of the proposed method over native XFormers across varying batch sizes and output sequence lengths, evaluated on a single NVIDIA H20 GPU.

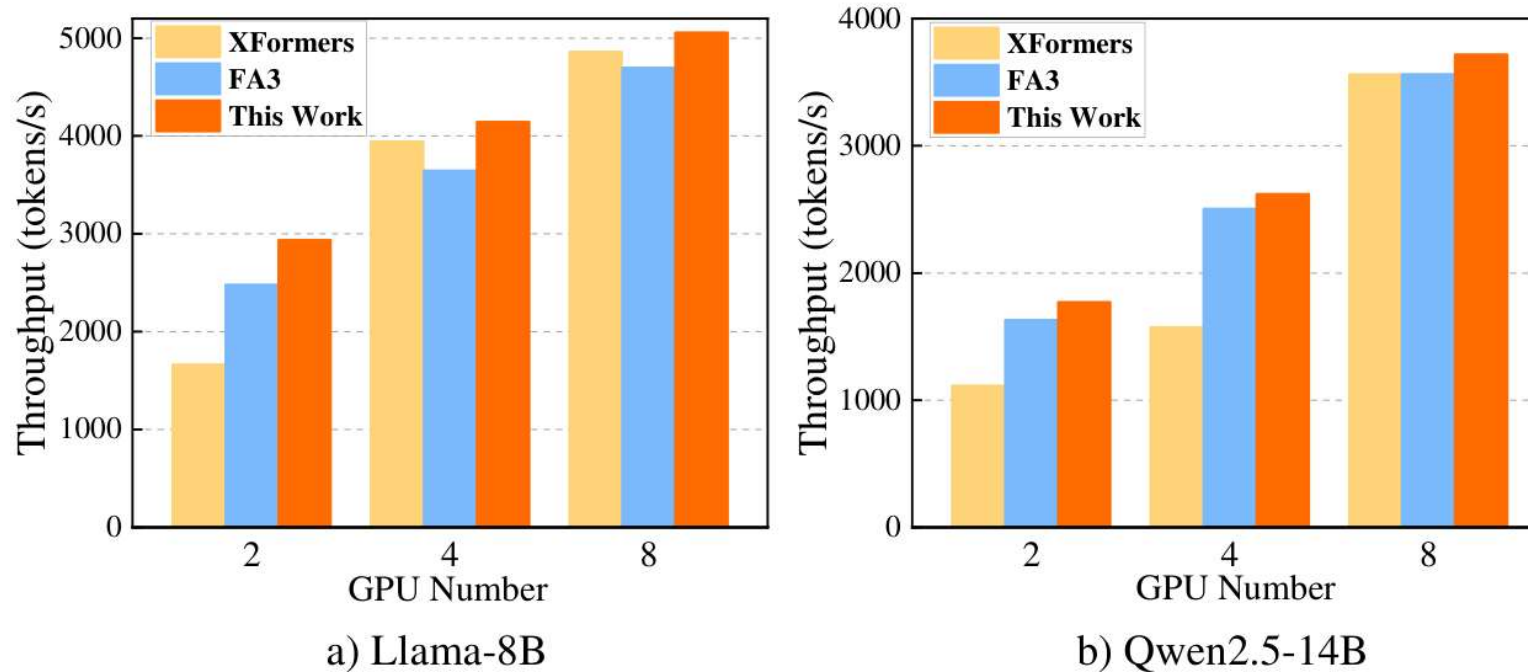


Figure 5: Multi-GPU end-to-end inference throughput comparison across backends under fixed 4096 output tokens with batch size 64, benchmarked on NVIDIA H20 GPUs.

Summary

Problem

- Suboptimal GPU Utilization
- Poor Cache Hit Rate
- Persistent Warp Stall

Solution

- load the next iteration's required K/V Blocks from high-latency HBM into L2 Cache.

Result

- achieves $2.15\times$ kernel acceleration and $1.97\times$ throughput improvement